

Real-Time Machine Learning-Based Threat Detection for ISP Network Traffic

June 2025

Abstract

The exponential growth of the Internet and the IoT devices induced a very high amount of new vulnerabilities, making cybersecurity field a top priority. The traditional anomaly detection based on fixed signatures favor latency and reduce the accuracy. In this research, the performance of supervised and unsupervised learning algorithms was tested for a binary classification task. A proof-of-concept prototype was build using Suricata for traffic extraction, Redis streams for managing the the traffic flux, BigBird for vector embeddings, a Random Forest classifier for making predictions, and an active learning loop for incorporating the expert's feedback to continuously refining the detection of new attack patterns. The results showed that supervised learning models, especially Random Forest Classifier, demonstrated high performance on the binary classification task.

1 Introduction

The Internet and the number of connected IoT devices have experienced, in the recent years, an exponential increase. With this increase, many vulnerabilities and weaknesses were induced, making the cybersecurity field a paramount priority. By shaping economic balance, tailoring social connections and developing public opinions, the network has a very important role in human life.

Due to the complexity of software and the evolution of cyber threats, it is necessary to acquire strong mechanisms to recognize, understand, and mitigate vulnerabilities. Firewalls and IDSes represent defense strategies that help in monitoring events to provide warnings or alerts about unusual traffic. To enhance the ability of IDSes to detect novel or previously unseen attacks, unsupervised learning machine learning models have gained popularity, particularly because they do not require labeled data and they could identify abnormal and unusual patterns in the network flow.

Researchers have extensively explored machine learning (ML) techniques and have adopted various detection methodologies. The main type of IDSes are, in general: model-based, signature-based, unsupervised and hybrid. Every one of these detection methods has its downsides, including the necessity of human interaction for data tagging, the inability to handle invisible attacks, and the necessity of frequent updates because the network traffic is always changing.

Additional Keywords and Phrases: IDS (Intrusion Detection Systems), anomaly detection, supervised learning, unsupervised learning, active learning, Big Bird, decision tree-based algorithms

1.1 Problem Statement

Bright Access is an internet service provider with its own proprietary network infrastructure. Leveraging in-house developed technologies, Bright Access enables the configuration, monitoring, and management of telephony, data communication, and fiber-optic networks. Partell is currently developing a system capable of real-time classification of network packets. This system will identify potentially malicious traffic by analyzing network behavior. Traditional detection methods rely on fixed signatures or pattern recognition after incidents occur, which introduces latency and reduces effectiveness. Partell seeks to improve upon these methods using machine learning (ML) models for proactive, intelligent threat detection.

This research has the goal of creating a transition from a passive conduit to an active guardian of internet security. It aims to investigate the trade-offs between different ML models in terms of accuracy and time efficiency and to create an innovative system that performs threat detection on ISP-level traffic.

Based on the problem statement, the following research questions were addressed.

1.2 Research Questions

RQ1: How do various supervised-learning algorithms perform in the binary classification task?

RQ2: How do various unsupervised-learning algorithms perform in binary classification task?

Section 2 introduces a literature overview of the relevant studies related to this research topic, Section 3 presents the methodological approach applied during this research. The findings of the research study are presented and discussed in Section 4. In the sections 5 and 6 are presented the design overview and the technical details of the prototype. The research paper ends with a conclusion and future research recommendations in the Section 7.

2 Literature Review

An IDS is a security service which monitors and check system events with the goal of identifying and offering real-time warnings regarding unauthorized access to the system's resources. There are many distinct types of intrusion detection systems, and they can be distinguished by their operational style. Nowadays, signature-based IDSes like Suricata [1] and SNORT [2] make up the majority of those utilized in production. With these solutions, security professionals are updating the IDS rules by create a signature for every new attack that is taken place. Consequently, only when an activity exhibits a pattern that corresponds to a known attack signature is it marked as abnormal.

Because of the shortcomings of signature-based intrusion detection systems, machine-learning based strategies were investigated.

These strategies rely on supervised learning models. Emad E. Abdallah et al.[3] explores the application of supervised learning in respect with IDSes. This category of IDSes use a supervised-learning model which has contain a knowledge base constructed on labeled data. The new activity is then classified into malicious or normal, essentially building the binary classification task, in which the model must assign one of the 2 class labels to the traffic packets.

In Anomaly-based IDSes, unsupervised learning algorithms such as grouping and identifying unusual data points are used, to discover uncommon patterns from typical actions and spot possible security breaches.

Analyzing the payload content of each network packet may be crucial for identifying abnormal traffic. Large data (payload) size can be a major obstacle for real-time predictions, as every millisecond counts in processing and transmitting information. The impact of latency introduced by large data (payload) volumes is a critical constraint in artificial intelligence architectures designed for high-frequency inference. In this paper, it was proved the fact that, when combined with the vector embedding technique, payload analysis leverage the detection of very complex attack within the transmitted content which may bypass the traditional signature-based approaches.

2.1 Limitations

One limitation may be induced by the lack of real labeled data. Most of the data sets represent simulations of different attacks. This has a negative impact on the detection of real-world attacks. Moreover, this negative impact may be crucial for model's training phase, leading to a packet missclassification. Another limitation is related to the signature-based IDSes. They operate with a set of implicit rules. Without expert's intervention, signature-based IDSes cannot identify new patterns in the network traffic. Whereas unsupervised learning present advantages in terms of identifying new attacks, their primary limitation is the high rate of false positives. Any deviation from the normal data is classified as a potential attack.

Table 1 presents an overview of some existing solutions.

Paper	Proposed solution	Description of the approach
Boukela et al.(2021)[4]	Hybrid IDS using Active Learning with DNN(supervised module) and KNN(unsupervised module)	The IDS processes the the network traffic in an incremental procedure using sliding window of certain size. It uses an active learning model constructed on a DNN (supervised module) architecture for identifying known pattern and KNN (unsupervised module) model for identifying unknown patterns. Through uncertainty sample query function, the AL model asks an expert to verify the label of the most uncertain predictions.
Amado et al.(2023)[5]	Malicious traffic detector using Kitsune’s KitNET neural network(unsupervised module)	The solution uses Perforce(P4) software to extract packets over the network switch. The neural network is built on autoencoders architecture. The performance of the solution is measured using the AUC metric. The threshold for considering a packet malicious is less than 0.5.
Nico Görnitz et al.(2009)[6]	ActiveSVDD: anomaly detection using active learning task using Support Vector Data Description	This approach uses hypersphere-based anomaly detection(semi-supervised learning). It combines 2 strategies: <i>Marginal Strategy</i> : implies examination of points closed to the decision hypersphere and <i>Exploration strategy</i> : determine attacks by exploring unknown areas in the feature space.
Paolini et al.(2024)[7]	Packet Classification Prototype using CNN-architecture	This approach collects every specific feature related to an individual packet and creates an matrix of size $n \times p$ where n is the number of features selected and p is the number of packets within a single flow. It uses a custom CNN architecture(supervised-learning module) for training and testing. It measures the performance of the model using the accuracy metric.

Table 1: Summary of existing solutions

3 Methodology

This section outlines the methodological approach used during this research. This includes the following:

1. *Data Collection and Feature Engineering*: Collecting, testing labeled datasets and selecting meaningful features for training, testing, and validating the model’s predictions.
2. *Vector Embeddings*: Create vector embeddings using the Big Bird transformer using natural language processing technique
3. *Supervised-learning*: test supervised learning models on the binary classification task
4. *Unsupervised-learning models*: test unsupervised learning models on the binary classification task

5. *Metrics*: evaluating model performance using classification report for supervised -learning models and silhouette score and accuracy for unsupervised-learning models

3.1 Data Collection and Feature Selection

Data collection represents a fundamental step that seeks to retrieve meaningful information to build a consistent dataset. The dataset used in this study contains network packet data, where each individual packet is represented in JSON format, extracted from PCAP files, using a custom Suricata plugin. This also helped in the data labeling process. For this research, 400 unique individual packets were used, 200 labeled as normal/benign, and 200 labeled as malicious.

An effective network traffic analysis depends on the ability to select meaningful features from raw data, allowing machine learning models to differentiate between anomalous and normal behavior. Feature selection plays a very important role in transforming raw data into structured representations appropriate for these models. Feature extraction at packet-level involves parsing individual network packets to recover entire IP header attributes and the packet payload hexadecimal represented.

3.2 Vector Embeddings

One of the fields of artificial intelligence and machine learning that is expanding the fastest is natural language processing. Its goal is to enable machines to comprehend natural language and to provide tools that help people complete tasks involving natural language. In order to facilitate this task, at first the data is necessary to be converted into a format which machine can understand. The method used in this research is vector embeddings, needed for making the correlation in between the raw data and the vector-space. Tokenization is the process of breaking down text into smaller parts in a way that is meaningful for the machine by keeping the same context. In this research, 2 types of tokenization are implied:

1. **Word tokenization**: this approach breaks down the text sentence into words; in this study, the JSON string representing the IP header content is split into individual words and converted into tokens
2. **Byte-pair encoding**: this approach yields to a fixed-size subword vocabulary; in this study, each 2 hex characters of the packet payload are converted into a token.

Big Bird is transformer-based model that introduces sparse-attention mechanism: a mechanism that modifies the quadratic complexity of models into linear, designed for handling long token sequences. Below, the architecture of the BigBird model is described:

Big Bird embeddings block:

1. **word_embeddings**: this layer is the input which needs to be fed into the model; the vocab size represents the amount of values a token can have; at this level each word is converted from its token array representation to its representative floating-point vector
2. **position_embeddings**: because the transformer doesn't include inherent knowledge about word order, the position embedding is added to each of the vectors from the previous layer, based on the position of each word in the sequence.
3. **tokens_type_embeddings**: this layer has the role of segmentation. The embedding is dependent on the the group the words belong to and is added to the previous layer embedding
4. **LayerNorm**: this layer applies normalization over all floats in the vector

Big Bird Encoder block:

At this level, the model has 4 block-sparse attention layers[8]. The attention mechanism projects the vector representation for each word into 3 new different vectors: query, key and value. Afterwards, the attention score is calculated by taking the dot product between the query vector of the current word and the key vectors for each other word. The scores are going through a softmax function which converts them into probabilities. The final vector is represented by the weighted sum of all value vectors.

3.3 Supervised Learning

Supervised learning is a machine-learning subdomain which utilizes labeled data in the training process of the models in order to identify common patterns or relations between the input and output. Classification represents a supervised-learning task which is characterized by predicting the label of a value. In this study, the binary classification task is used for trying to answer the question whether a packet is malicious or not.

Decision-Tree algorithms are algorithms made for classifying data by making sequential decisions at each node, each node being a test on a feature, guiding data down on different branches until the classification is done.

1. **Random Forest**: represents an algorithm which uses several decision trees to train random subsets of data and outputs the prediction based on majority voting.
2. **Gradient Boosting**: represents an algorithm which creates several decision trees sequentially. Unlike random forest, Gradient Boosting is constructing each new decision tree to correct the residuals generated by the preceding trees.

Logistic Regressor represents an algorithm that returns the probability that an input value is part of a class. If the prediction value is smaller than 0 then the logistic regressor returns the value -1 (the value is labeled with class 0). Otherwise, it returns the value 1 (the value is labeled with class 1).

3.4 Unsupervised Learning

Unsupervised learning is a machine-learning subdomain which deals with unlabeled data. Unlike supervised learning, unsupervised learning algorithms strive to identify common or unusual patterns within the raw data by grouping similar data points or finding outliers, without predefined labels.

Clustering

1. **KMeans**: is a clustering algorithm used for grouping similar data points together. It starts by selecting a value k which is related to the amount of groups (centroid)s data needs to be classified in. The data point is assigned to the closest centroid based on a distance metric. (see Section 3.5.3)
2. **Agglomerative clustering**: is a hierarchical clustering algorithm. At first, each data point is treated as an individual cluster. Then it iteratively merges clusters together, gradually forming larger clusters, until every data point are grouped into one cluster or a stopping criterion is seen.

Anomaly detection

1. **SVM** (Support vector machines): OneClassSVM is an unsupervised or semi-supervised algorithm used for anomaly detection. It is trained only on data from the "normal" class, learning a decision boundary that encompasses these benign data points. During prediction, any new data point falling outside the learned boundary is considered an anomaly or potentially malicious instance.
2. **Isolation Forest**: is an algorithm which repeatedly splits the data and isolates the anomalous data points from the normal ones.

3.5 Metrics

3.5.1 Classification Report

Classification report is a function which summarizes the metrics used to evaluate the performance of the models in the classification task. It covers the following metrics: accuracy, precision, recall and f1-score. Interpretation of the metrics:

1. **Accuracy** represents the proportion of the corrected predictions made by the model
2. **Precision** proportion of model's all positive predictions which were actually positive.
3. **Recall** represents the proportion of all predictions which have a true label positive classified correctly as a positive.
4. **F1-Score** the proportion of predictions with true label of negative which were misclassified

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

$$F1 - Score = \frac{2 * TP}{2 * TP + FP + FN} \quad (4)$$

3.5.2 Silhouette Score

Silhouette score¹ is metric which is used for evaluating the performance of a clustering technique. The silhouette score metric is calculated using the formula:

$$silhouette_score = \frac{b - a}{\max(a, b)} \quad (5)$$

where a is the average distance between each point within a cluster, or the average intra-cluster distance b is the average distance between all clusters, or the average inter-cluster distance. The Silhouette score has a range of -1 to 1. A cluster is dense and well-separated from neighboring clusters if its score is 1. Overlapping clusters with samples extremely close to the surrounding clusters' decision border are represented by a value close to 0. The samples may have been assigned to the incorrect clusters if the score is negative [-1, 0].

3.5.3 Cosine Distance

Cosine similarity quantifies the degree of similarity between two vectors in an inner product space. It determines whether two vectors are pointing in about the same direction and is calculated by taking the cosine of the angle between them. It is often used in text analysis to measure the similarity between documents. Cosine distance calculates the cosine of the angle between two vectors to determine how dissimilar they are.

$$cosine_similarity = \frac{\langle X, Y \rangle}{(||X|| * ||Y||)} \quad (6)$$

$$cosine_distance = 1 - cosine_similarity \quad (7)$$

where $\langle X, Y \rangle$ is the dot product of vectors X and Y, $||X||$ = the magnitude of the vector X and $||Y||$ is the magnitude of the vector Y. The resulting cosine similarity score indicates how similar two vectors are in terms of direction. A score of 1 implies that the vectors are aligned; 0 indicates that they are orthogonal (perpendicular), and -1 indicates that they are diametrically opposing.

4 Results

4.1 Experiment 1

This experiment consists in evaluating the performance of different supervised-learning models on the binary classification task. The tested models are Random Forest Classifier², Decision Tree Classifier³, Gradient Boosting Classifier⁴ and Logistic Regression⁵. After testing the models with the synthetic datasets, the results shown a sign of overfitting. The accuracies on the train dataset very high and the accuracies on the test sets very extremely low. This didn't happen when testing the dataset described in section 3.1. Tabel 2 and 3 show the overview of the results. These results and hyper-parameters are impacted by the size of the datasets.

¹sklearn.metrics.silhouette_score

²sklearn.ensemble.RandomForestClassifier

³sklearn.tree.DecisionTreeClassifier

⁴sklearn.ensemble.GradientBoostingClassifier

⁵sklearn.linear_model.LogisticRegression

Supervised Learning Model	Best Hyperparameters	Accuracy
RandomForestClassifier	'bootstrap': False, 'criterion': 'entropy', 'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200	95
DecisionTreeClassifier	'criterion': 'entropy', 'max_depth': 10, 'min_samples_leaf': 4, 'min_samples_split': 10	85.31
GradientBoostingClassifier	'learning_rate': 0.2, 'max_depth': 10, 'n_estimators': 100, 'subsample': 0.8	94.69
Logistic Regression	'C': 1, 'max_iter': 100, 'penalty': 'l2', 'solver': 'lbfgs'	72

Table 2: Supervised learning models with the best Hyperparameters and corresponding accuracies

Supervised Learning Model	Accuracy	Precision		Recall		F1-Score	
		Normal	Malicious	Normal	Malicious	Normal	Malicious
RandomForestClassifier	93	96	91	89	96	92	93
DecisionTreeClassifier	84	90	78	89	89	84	83
GradientBoostingClassifier	91	92	89	89	93	90	91
Logistic Regression	72	79	67	68	78	73	72

Table 3: Performance metrics of the supervised-learning models

4.2 Experiment 2

Similar to Experiment 1, unsupervised-learning models were evaluated. The tested models are: KMeans, Isolation Forest, Agglomerative Clustering and OneClassSVM. Because the used dataset was labeled, the accuracy could be calculated. Table 3 shows an overview of the accuracy result for all models. Verifying the silhouette score was 0.34 for the KMeans clustering, which means that the classes were overlapping each other, which results that some data was misclassified.

Model	Type	Accuracy
KMeans	Clustering	57.25
Agglomerative Clustering	Clustering	55.5
Isolation Forest	Outlier detection	42.5
OneClassSVM	Outlier detection	55

Table 4: Unsupervised-learning models with the best Hyper-parameters and corresponding accuracies

5 Design

The UML Diagram (Figure 1) shows the workflow of the system. At first, Suricata is used to filter and extract the content of an individual packet. Then, the content is parsed to a Redis stream, which acts like a message queue. Once data passes the message queue, a transformer is used to convert the data into a vector embedding. The vector embedding is fitted into a trained random forest model which returns the prediction. In this design, an active learning loop is included. The predicted value needs to be checked and corrected by an expert. All corrected data is saved to a PostgreSQL (with pgvector plugin) vector database which plays the role of a data warehouse. The data manipulation process is made with the help of SQL queries.



Figure 1: UML diagram of the design

6 Prototype

For building the prototype, the following open-source and licensed software tools were used:

1. **Suricata** - an open-source signature-based IDS/IPS (intrusion detection system/intrusion prevention system) which was used for retrieving packet metadata from the network traffic. This generates a JSON log which includes IP addresses, ports, etc. Because Suricata doesn't provide, in its standard configuration, the raw metadata, for example the payload in hexadecimal representation, it was necessary to make a custom plugin in C language. The custom plugin sends a JSON object consisting of the IP header and the hex payload of an individual packet to the Redis stream.
2. **Redis Stream** - represents a tool designed for linking data streams and the machine learning model. It has the feature to store, access and read streams of messages. Redis offers a very important feature for handling high-throughput traffic called consumer groups. To deal with high throughput and low-latency, it allows creating multiple working threads which can read the data simultaneously.
3. **Transformer** - The prototype uses a NLP approach using the Big Bird transformer for embedding large text sentences. This is chosen because analysis on high-volume payloads are performed. The model contains special tokens which are used for truncation, padding and storing the vector embedding. The vector embedding is stored in the CLS token in the last hidden state of the transformer.
4. **Supervised Learning and active learning loop** - The prototype uses a random forest classifier to make binary classification. The model was chosen based on the accuracy performance seen in the experiments. Since supervised learning models present a fixed knowledge base when it comes to identifying patterns, an active learning loop gives the model opportunity to grow by leveraging expert feedback in

order to identify new patterns and refine the predictions over time. After the model makes the predictions, an expert can correct the labels and can store the corrected data in a vector-base database. Database manipulation is done with the help of SQL queries: insertion of the corrected data and selection of the historical data for re-training the model.

5. **Streamlit** - python UI framework. It was used in combination with HTML and CSS to display the real time predictions and the performance of the random forest model. The prototype is split in 2 pages: the first page is used for showing and correcting the labels from the live data and the second page is used for model training.

7 Conclusion and Future Work

The results are showing that identifying malicious network packets using a supervised-learning model like random forest can be feasible. In this context, the unsupervised learning models demonstrated one of their limitation, being prone to false positives. The proposed prototype serves as a proof-of-concept, is constrained by resource limitations, focusing on leveraging viability rather than functionality at a production level.

1. **Offline database:** The actual databases which were used, were deployed locally inside docker containers in the VM. A scaled solution for handling the high throughput would be the migration of the offline database to an AWS environment so that the system would benefit from a dynamical resource allocation.
2. **Message streams and group of redis consumers:** currently the prototype uses only 1 data stream and a group of 3 Redis consumers. A scaled solution for this may be splitting the network traffic across multiple VM running the prototype. This would imply creation of multiple data streams and groups of consumers.
3. **Front-end:** the prototype uses the streamlit python framework. A better alternative would be a front-end made with a JavaScript framework such as React, Vue.js or Angular.

REFERENCES

- [1] Suricata-IDS, <https://suricata-ids.org/>
- [2] M. Roesch, "Snort - Lightweight Intrusion Detection for Networks," In Proceedings of the 13th USENIX conference on System administration (LISA '99), 1999, USENIX Association, pp. 229–238.
- [3] Abdallah, E. E., Eleisah, W., Otoom, A. F. (2022). Intrusion Detection Systems using Supervised Machine Learning Techniques: A survey. *Procedia Computer Science*, 201, 205–212. <https://doi.org/10.1016/j.procs.2022.03.029>
- [4] Boukela, L., Zhang, G., Yacoub, M., Bouzefrane, S. (2021). A near-autonomous and incremental intrusion detection system through active learning of known and unknown attacks. 2021 International Conference on Security, Pattern Analysis, and Cybernetics (SPAC). <https://doi.org/10.1109/spac53836.2021.9539947>
- [5] Amado, J. R., Pereira, F., Pissarra, D., Signorello, S., Correia, M., Ramos, F. M., V. (2024). Peregrine: ML-based malicious traffic detection for terabit networks. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2403.18788>
- [6] Nico Görnitz, Marius Kloft, Konrad Rieck, and Ulf Brefeld. 2009. Active learning for network intrusion detection. In Proceedings of the 2nd ACM workshop on Security and artificial intelligence (AISeC '09). Association for Computing Machinery, New York, NY, USA, 47–54. <https://doi.org/10.1145/1654988.1655002>
- [7] Paolini, E., Valcarengi, L., Maggiani, L., Andriolli, N. (2024). Real-Time network packet classification exploiting computer vision architectures. *IEEE Open Journal of the Communications Society*, 5, 1155–1166. <https://doi.org/10.1109/ojcoms.2024.3363082>
- [8] Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., Ahmed, A. (2020). Big Bird: Transformers for longer sequences. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2007.14062>